Snake n' Shake Team 8's Proposal

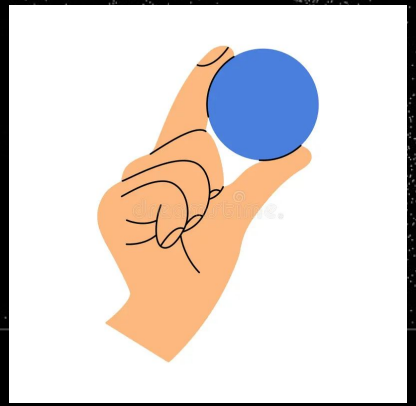
Amer Salem, Anirudh Kompella, Munasib Ilham, and Chance Howarth

Spring 2026 / ECE 554

The Team

Member	Major	Key Skills	Primary Responsibility
Chance Howarth	CE	FPGA, hardware design	Set up hardware infrastructure (VGA, SDRAM etc) Color detection and gesture recognition from the camera feed
Munasib Ilham	CE	C/C++, Verilog	Snake engine - Display rendering and snake logic
Anirudh Kompella	CS	C/C++, React+JS	Set up hardware infrastructure (VGA, SDRAM etc), testing and project report
Amer Salem	CE	FPGA, verilog	Top-level wiring, direction-detection, Hardware testing

Camera Controlled Snake Game



1 What and How

2 The Stack

3 Visual Mapping

4 The Logic

5 The Timeline

6 Risk Mitigation

A Real Time Snake Game

Traditional FPGA demos are often static or button based, and do not show real-time interactive computer vision.

- Need for real-time interaction
- Pipelining camera input with decision making and graphics all in one.
- Tight timing constraints requiring efficient parallel hardware design

We implement a real-time Snake game entirely in hardware on FPGA, utilizing a camera that captures the player's movement.

- Hardware camera pipeline
- Gesture capturing module
- Game logic module
- Rendering Module
- Shared Memory
- Game state machine

Our Solution

Must Haves

1. Camera Capture
2. Color Detection
3. Snake Game
4. VGA Display Output
5. Gesture Based Direction Control
6. Live Game Feedback (Score, state, highscore)

Nice to Haves

1. Difficulty scaling
2. Config Options
3. Keyboard/mouse support
4. Audio Output

The Stack - System Level Components

Hardware Platform

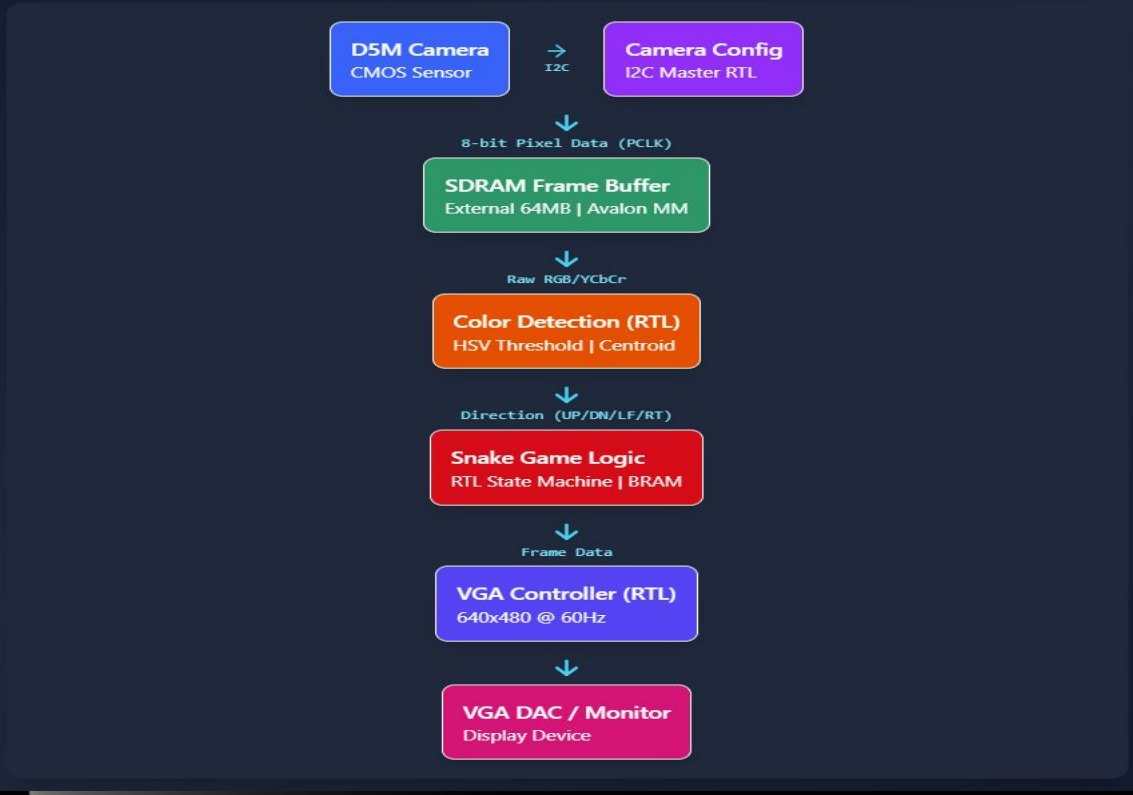
- DE1-SoC board
- Camera
- VGA/Display Output
- On-board Memory (Frame buffering)
- Push buttons, switches, and display for fall back controls and debugging.

Hardware Modules

- Gesture Detection (direction signal)
- Game Logic (collision, movement)
- Display Rendering
- Fully hardware based, no processor used

D5M Camera-Controlled Snake Game System

System Architecture



Core Algorithms & Optimizations

Gesture Detection Algorithm

- Process incoming frames pixel-by-pixel
- Detect predefined color range
- Divide screen into four zones
- Determine the dominant zone
- Filter rapid changes to avoid accidental inputs

Snake Game Logic

- Grid-based state machine
- Each tick, snake position is updated
- Collision Detection - self and wall
- Random food placement
- Score and growth handling

Display Rendering

- Raster-scan style pixel generation
- Where each pixel belongs
- Score and UI
- Output directly to VGA pipeline

I/O Devices

Device	Interface	Purpose
Camera	Parallel + SCCB (I2C)	Player input — color detection
VGA Monitor	VGA	Game output
DE1-SoC 7-segment display	GPIO	Score output
Push buttons	GPIO	Reset, pause, movement

Week	Task / Milestone	Owner	Description
Wk 1-2	Setup & hardware bring-up	All	Verify camera, display, and memory work on the board; divide module ownership
Wk 2-4	Gesture detection	M1	Detect the colored object from camera and output a direction signal
Wk 3-5	Snake game logic	M2	Movement state machine, collision detection, and food placement
Wk 4-5	Display rendering	M3	Draw the snake, food, borders, and score onto the screen
Wk 6	**Midterm checkpoint**	**All**	**All modules verified in simulation; Snake playable via push-buttons on FPGA**
Wk 6-8	Synthesis & wiring	M1-M4	Compile all modules to hardware and wire everything together
Wk 8-9	Hardware testing & debug	All	Test on the board; fix timing issues; tune gesture detection
Wk 10	**Final demo + report**	**All**	**Fully playable camera-controlled Snake on FPGA; report submitted**

Risks

- Unreliable Gesture Detection under different environments.
- Game logic does not synthesize correctly in hardware
- Timing closure fails after full integration
- Running out of on-chip memory for the snake body
- Modules not working properly when integrated

Mitigation

- Adjustable sensitivity threshold via switches
- Fully test all logic in software simulation prior to compiling
- Consistently setting up constraints and testing periodically
- Add limits to the game states, use external memory if needed
- Weekly check-ins, verify integrity of each module before integration.